

Lab Work 02

Topic	Encapsulation, Access Modifiers, Getters/Setters, Properties, Name Mangling
Objective	Apply encapsulation principles to protect and manage access to object data using Python conventions.
AI Copilot Usage	Not Allowed

Tasks

1. Apply Naming Conventions:

- Modify the ***Student*** class from Session 1.
- Make the ***student_id*** attribute "private" using the double underscore prefix (`__`).
- Make ***first_name*** and ***last_name*** "protected" using the single underscore prefix (`_`). Keep ***units_enrolled*** public for now.
- Try accessing the ***__student_id*** directly from outside the class (e.g., `print(student1.__student_id)`). Observe the ***AttributeError***.
- Access the name-mangled version (***_Student__student_id***) to understand how it works (but emphasize this is bad practice for external access).

2. Implement Getters and Setters:

- Write a public getter method ***get_student_id()*** that returns the value of ***__student_id***.
- Write public getter methods ***get_first_name()*** and ***get_last_name()*** for the protected attributes.
- Write a public setter method ***set_last_name(self, new_last_name)*** that allows changing the ***_last_name***.
- Modify ***get_full_name*** and ***display_details*** to use the getter methods internally where appropriate (e.g., `self.get_student_id()` instead of `self.student_id` if it were public).
- Test these new methods from outside the class.

3. Use Properties (Pythonic Getters/Setters):

- Define a new class ***Teacher*** with a private attribute ***__employee_id*** and a protected attribute ***_subject_taught***.
- Implement a *property* for ***employee_id*** that allows reading the ***__employee_id*** but *not* setting it from outside (read-only property). Use the ***@property*** decorator.

- c) Implement a *property* for ***subject_taught*** that allows both getting and setting the ***_subject_taught***. Use ***@property*** and ***@subject_taught.setter***.
 - d) Instantiate a ***Teacher*** object and test accessing and modifying the properties.
4. **Creative Challenge (Optional, if time permits):**
- a) Add validation to the ***subject_taught*** setter in the ***Teacher*** class. For example, ensure the subject is not an empty string or is from a predefined list of valid subjects. If validation fails, print an error message and don't change the attribute.
 - b) Implement a property for the ***Student's full_name*** that dynamically generates the name using the protected ***_first_name*** and ***_last_name*** attributes when accessed. Make this a read-only property.

Important Notes

- This is an individual task. All code must be your own original work.
- Ask questions and seek help from the lecturer or teaching assistants if you are stuck.
- Focus on writing clean, well-organized, and well-documented code.